

Table 2

Mortality Rate and AAPC for Small Bowel Neoplasms Among U.S. Adults Aged 25 Years and Older

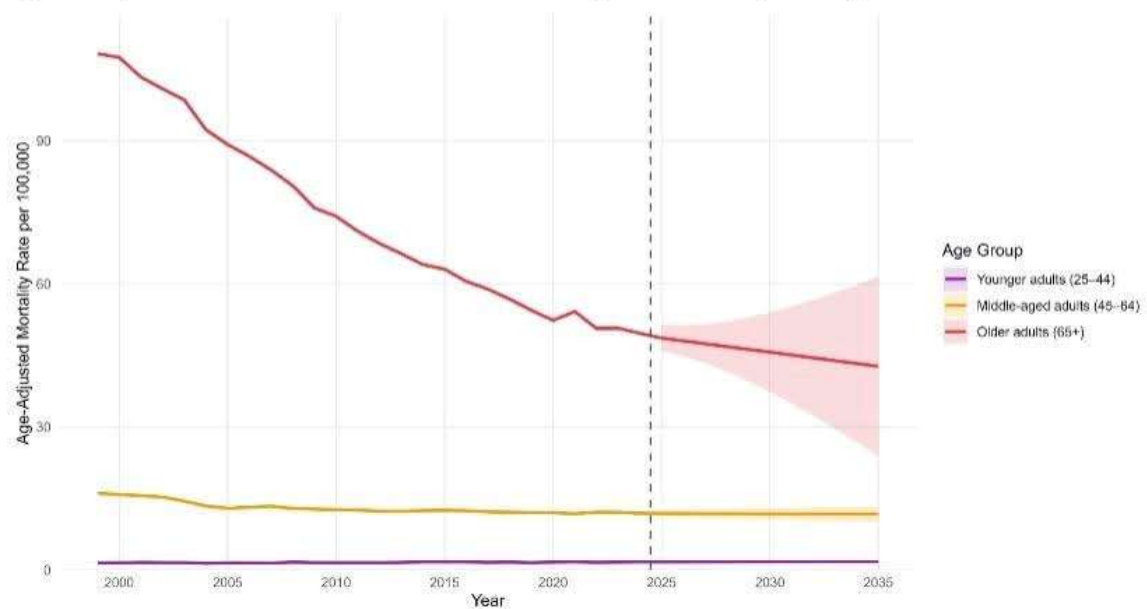
Demographics	Deaths	Population	Mortality Rate per 100,000 (95% CI)	AAPC (95% CI)
Overall	36,078	5,398,746,349	0.60 (0.58– 0.63)	1.00* (0.68 to 1.32)
Sex				
Male	19,475	2,605,943,716	0.74 (0.71– 0.77)	0.91* (0.57 to 1.24)
Female	16,603	2,792,802,633	0.51 (0.49– 0.54)	0.87* (0.42 to 1.32)
Race				
American Indian or Alaska Native	138	49,853,431	0.41 (0.34– 0.48)	—
Asian or Pacific Islander	833	245,576,124	0.43 (0.40– 0.46)	—
Black or African American	5,065	546,447,527	1.06 (1.03– 1.09)	—
White	25,929	3,631,977,407	0.64 (0.64– 0.65)	—
Age Group				
Younger adults (25– 44)	1,231	2,211,398,678	0.05 (0.05– 0.05)	0.00 (0.00 to 0.00)
Middle-aged adults (45– 64)	9,575	2,024,799,922	0.45 (0.43– 0.47)	0.21 (-0.30 to 0.71)
Older adults (65+)	25,272	1,162,547,749	2.18 (2.08– 2.29)	1.34* (1.04 to 1.65)
Census Region				
South	11,351	1,652,256,217	0.69 (0.67– 0.70)	—
West	6,670	1,024,837,182	0.65 (0.64– 0.67)	—
Midwest	7,798	969,567,311	0.80 (0.79– 0.82)	—
Northeast	6,146	827,193,779	0.74 (0.72– 0.76)	—
Urbanization				
Rural	4,746	678,634,169	0.58 (0.56– 0.60)	—
Urban	23,606	3,795,213,822	0.60 (0.59– 0.61)	—

Note. Mortality rates are per 100,000. Overall, sex, and age-group estimates cover 1999– 2024; race, census-region, and urbanization estimates cover 1999– 2020. Census-region entries are crude rates, whereas race and urbanization entries are age-adjusted mortality rates (AAMRs). AAPC = average annual percent change; CI = confidence interval.

* $p < .05$ for the log-linear AAPC.

Figure 1

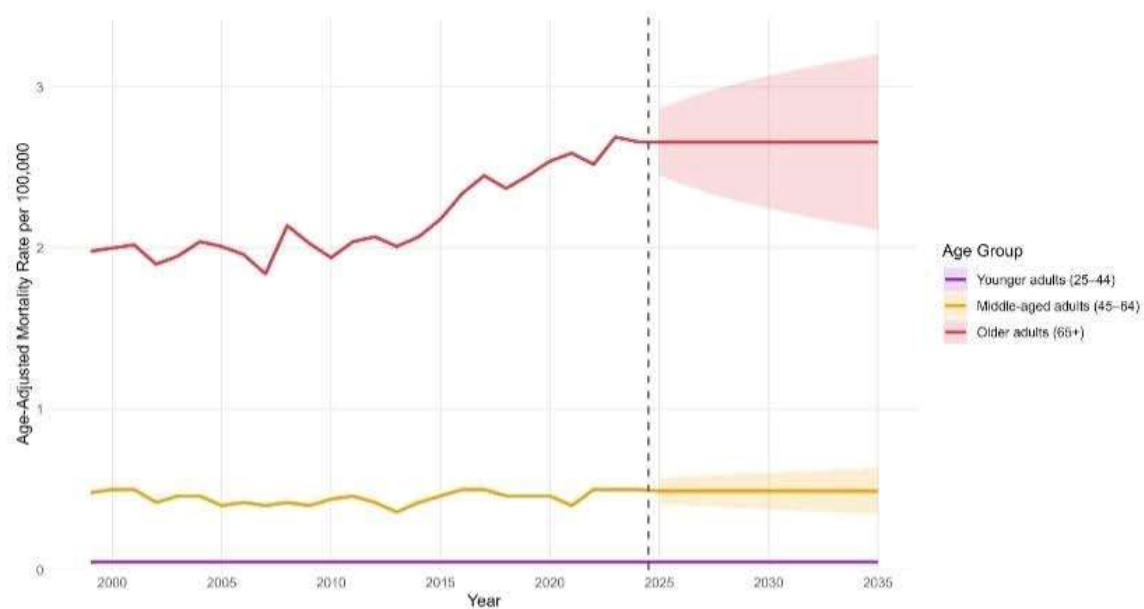
Age-Group AAMR Trends and Forecasts for Large Bowel Neoplasms, 1999– 2035



Note. Solid lines show observed age-adjusted mortality rates through 2024, and shaded bands show forecast uncertainty for 2025– 2035. The dashed vertical line marks the start of the forecast period. Forecast models were selected separately for each age group using the lowest rolling-validation RMSE.

Figure 2

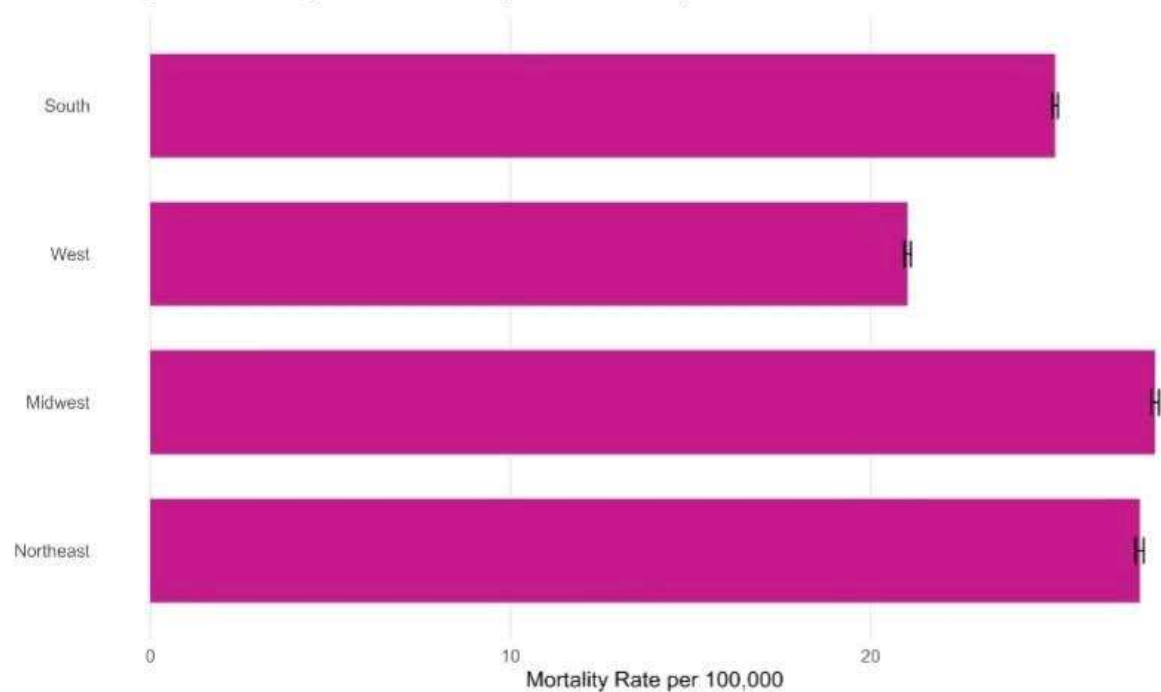
Age-Group AAMR Trends and Forecasts for Small Bowel Neoplasms, 1999– 2035



Note. Solid lines show observed age-adjusted mortality rates through 2024, and shaded bands show forecast uncertainty for 2025– 2035. The dashed vertical line marks the start of the forecast period. Forecast models were selected separately for each age group using the lowest rolling-validation RMSE.

Figure 3

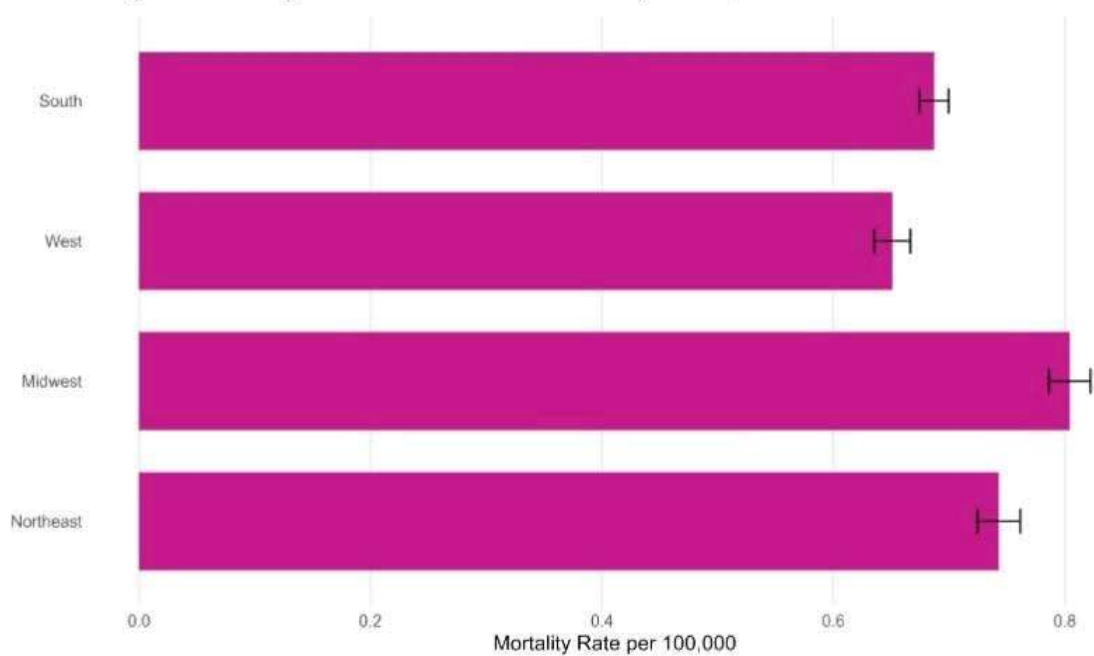
Census-Region Mortality Rates for Large Bowel Neoplasms, 1999– 2020



Note. Bars show period mortality-rate estimates, and error bars show 95% confidence intervals. Census-region entries are crude rates. AAPC is not displayed because annual regional series were not supplied.

Figure 4

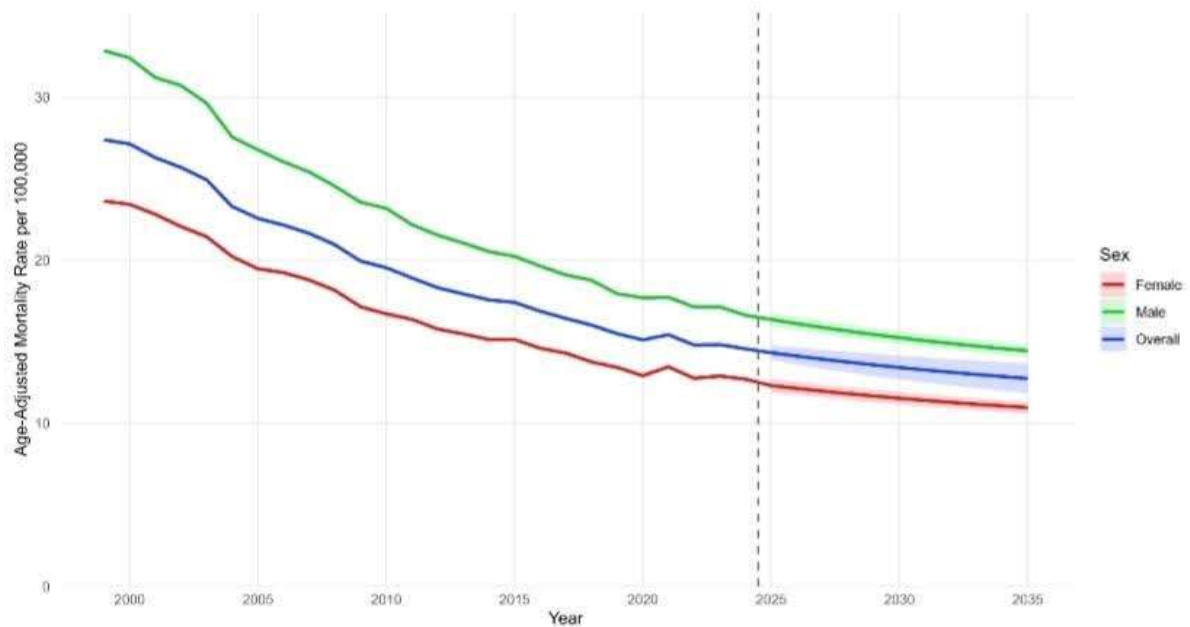
Census-Region Mortality Rates for Small Bowel Neoplasms, 1999– 2020



Note. Bars show period mortality-rate estimates, and error bars show 95% confidence intervals. Census-region entries are crude rates. AAPC is not displayed because annual regional series were not supplied.

Figure 5

Overall and Sex-Specific AAMR Trends and Forecasts for Large Bowel Neoplasms, 1999–2035



Note. Solid lines show observed age-adjusted mortality rates through 2024, and shaded bands show forecast uncertainty for 2025–2035. The dashed vertical line marks the start of the forecast period. Forecast models were selected separately for each series using the lowest rolling-validation RMSE.

Figure 6

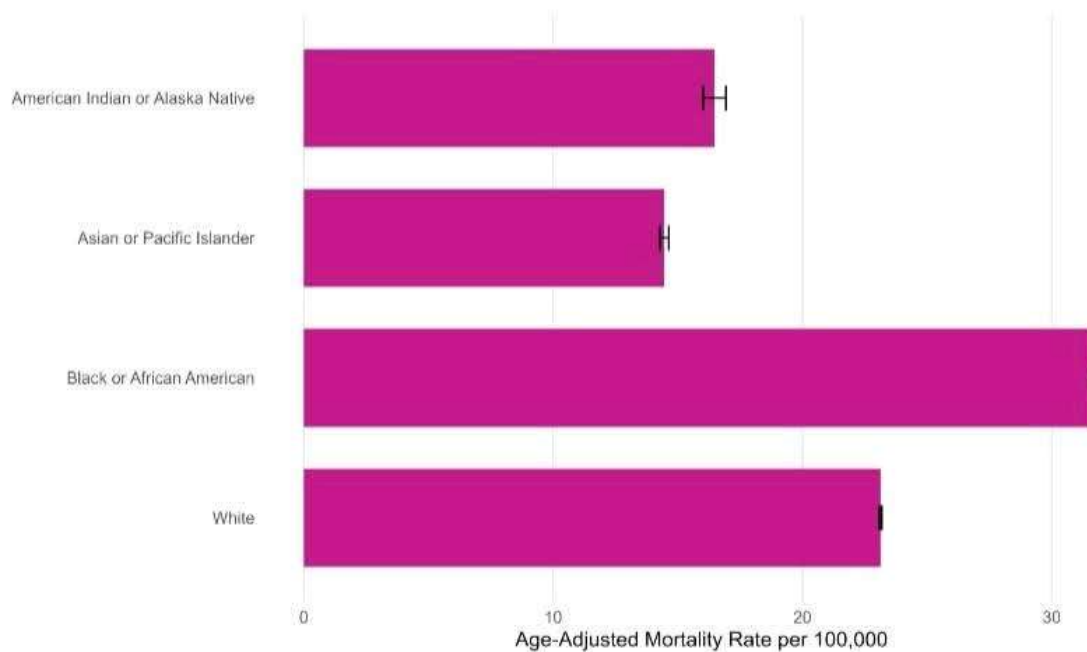
Overall and Sex-Specific AAMR Trends and Forecasts for Small Bowel Neoplasms, 1999–2035



Note. Solid lines show observed age-adjusted mortality rates through 2024, and shaded bands show forecast uncertainty for 2025–2035. The dashed vertical line marks the start of the forecast period. Forecast models were selected separately for each series using the lowest rolling-validation RMSE.

Figure 7

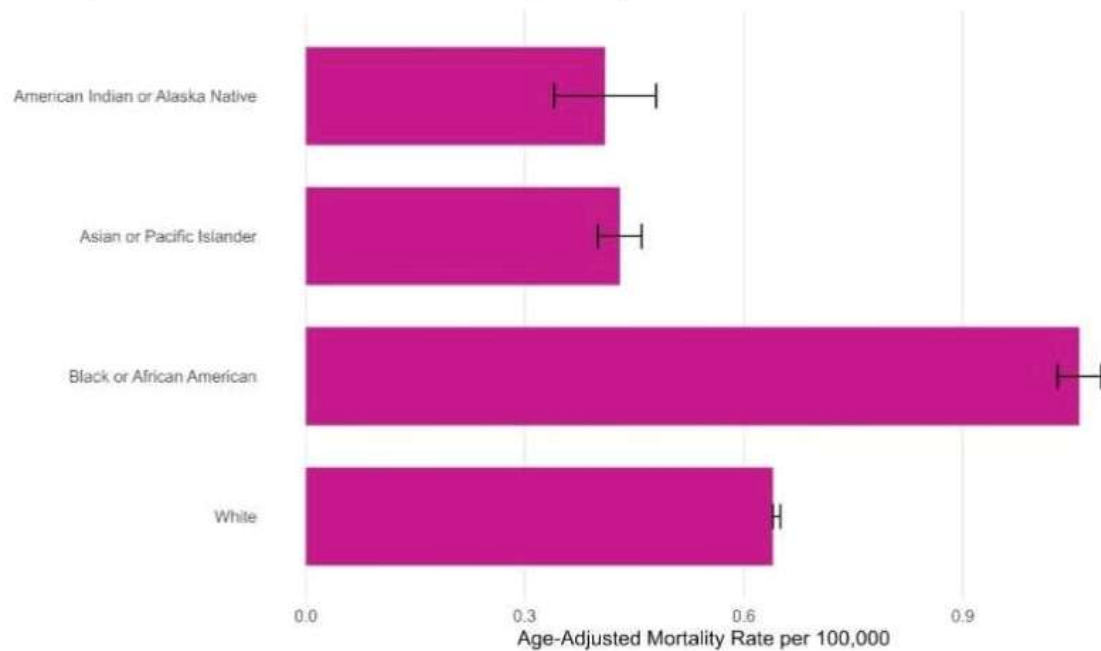
Race-Specific AAMRs for Large Bowel Neoplasms, 1999– 2020



Note. Bars show period age-adjusted mortality rates, and error bars show 95% confidence intervals. AAPC is not displayed because annual race-specific AAMR series were not supplied.

Figure 8

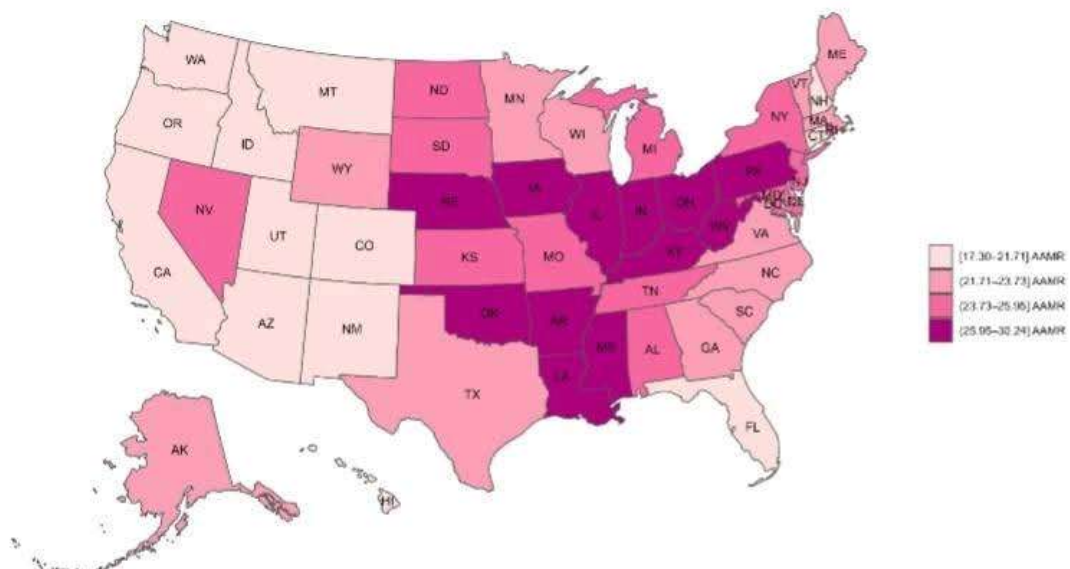
Race-Specific AAMRs for Small Bowel Neoplasms, 1999– 2020



Note. Bars show period age-adjusted mortality rates, and error bars show 95% confidence intervals. AAPC is not displayed because annual race-specific AAMR series were not supplied.

Figure 9

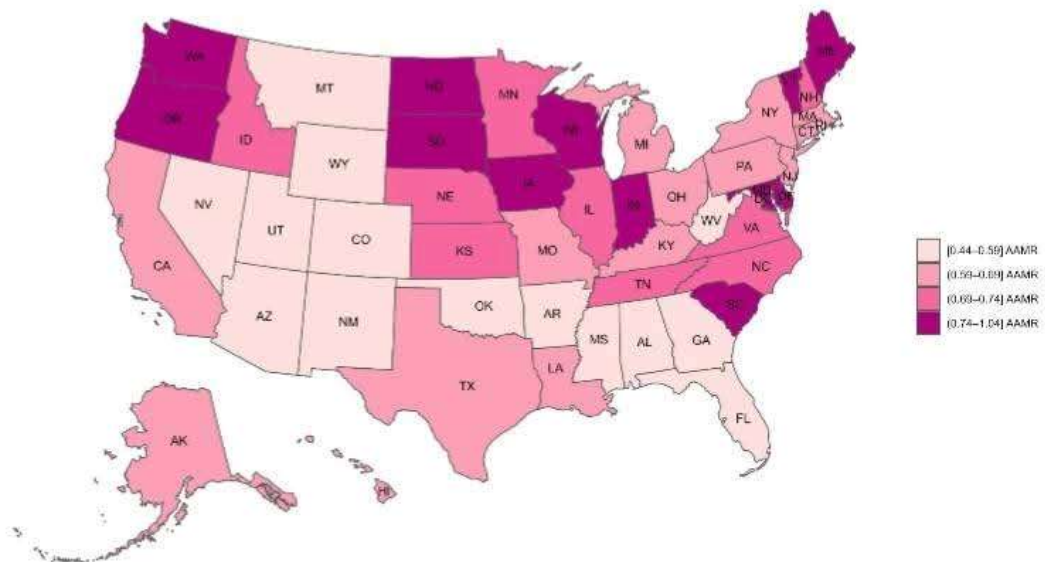
State-Level AAMR for Large Bowel Neoplasms, United States, 1999–2020



Note. States are grouped into four categories based on state-specific quartiles of the age-adjusted mortality rate per 100,000.

Figure 10

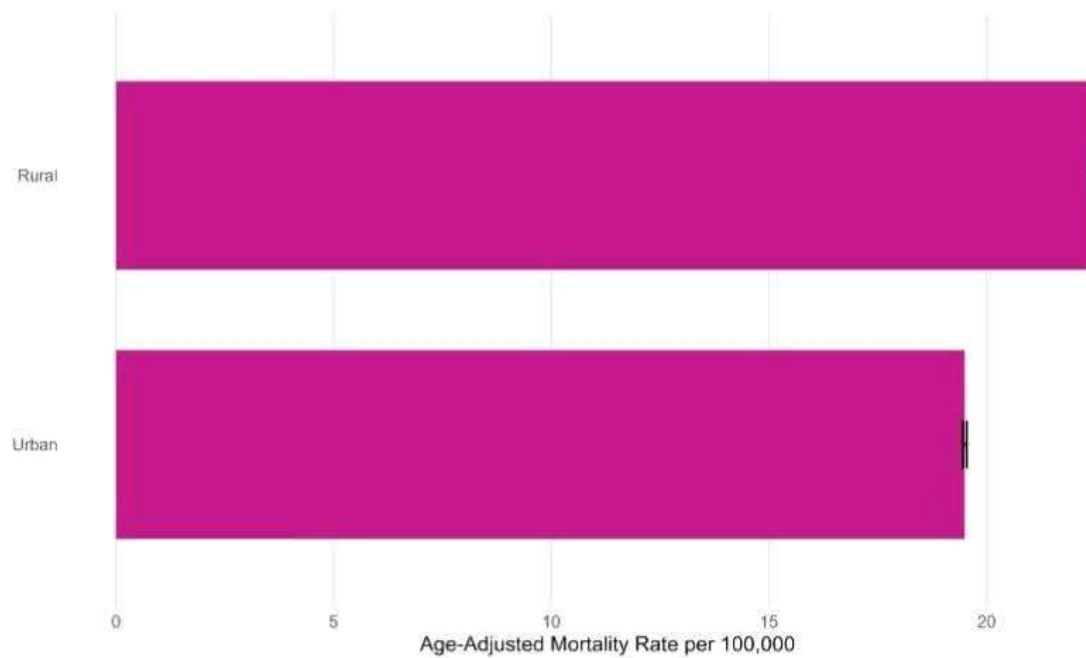
State-Level AAMR for Small Bowel Neoplasms, United States, 1999– 2020



Note. States are grouped into four categories based on state-specific quartiles of the age-adjusted mortality rate per 100,000.

Figure 11

Urbanization-Specific AAMRs for Large Bowel Neoplasms, 1999– 2020



Note. Bars show period age-adjusted mortality rates, and error bars show 95% confidence intervals. AAPC is not displayed because annual urbanization-specific AAMR series were not supplied.

Figure 12

Urbanization-Specific AAMRs for Small Bowel Neoplasms, 1999– 2020



Note. Bars show period age-adjusted mortality rates, and error bars show 95% confidence intervals. AAPC is not displayed because annual urbanization-specific AAMR series were not supplied.



CISCO ISR ROUTER

WHAT IS CIDR?



CIDR (Classless Inter-Domain Routing) is a method used to allocate IP addresses and create flexible subnet sizes without relying on Class A, B, or C boundaries.

EXAMPLE

192.168.1.0/24

Where:

- 192.168.1.0 = Network Address
- /24 = Prefix Length

UNDERSTANDING CIDR PREFIXES

CIDR	SUBNET MASK	BINARY (Network / Host Portion)
/24	255.255.255.0	11111111. 11111111. 11111111. 00000000
/25	255.255.255.128	11111111. 11111111. 11111111. 10000000
/26	255.255.255.192	11111111. 11111111. 11111111. 11000000
/27	255.255.255.224	11111111. 11111111. 11111111. 11100000
/28	255.255.255.240	11111111. 11111111. 11111111. 11110000
/29	255.255.255.248	11111111. 11111111. 11111111. 11111000
/30	255.255.255.252	11111111. 11111111. 11111111. 11111100

 Network Portion

 Host Portion

CIDR HOST CALCULATION

FORMULA

Usable Hosts = $2^{(32-\text{Prefix})} - 2$

/24  Total Hosts : 256
Usable Hosts : **254**

/25  Total Hosts : 128
Usable Hosts : **126**

/26  Total Hosts : 64
Usable Hosts : **62**

WHY CIDR IS IMPORTANT?



Efficient IP Address Utilization



Reduces IP Waste



Supports Route Summarization



Improves Routing Efficiency



Essential for IPv4 Networks



CIDR EXAMPLES

SMALL OFFICE

192.168.10.0/27

Usable Hosts: 30



MEDIUM OFFICE

192.168.20.0/26

Usable Hosts: 62



LARGE DEPARTMENT

192.168.30.0/24

Usable Hosts: 254



CIDR vs CLASSFUL ADDRESSING

CLASSFUL

- ✗ Fixed Network Sizes
- ✗ Wastes IP Addresses
- ✗ Less Flexible

VS

CIDR

- ✓ Flexible Subnet Sizes
- ✓ Better IP Utilization
- ✓ Modern Standard

KEY TAKEAWAY



CIDR =
Flexible
Subnetting



Uses Prefix
Length
(/24, /25, /26)



Saves IP
Addresses



Enables Route
Summarization



Foundation of
Modern IPv4
Networking

QUICK MEMORY TRICK

PREFIX	USABLE HOSTS
/24	254
/25	126
/26	62
/27	30
/28	14
/29	6
/30	2



Remember:
As prefix increases
by 1, hosts are
halved.

Linux Administration Ultimate Guide

Table of Contents

1. Introduction to Linux Administration

- 1.1 What is Linux Administration?
- 1.2 Why Learn Linux Administration?
- 1.3 Linux Distributions Overview
- 1.4 The Linux Philosophy
- 1.5 Roles and Responsibilities

2. Getting Started with Linux

- 2.1 Accessing a Linux System
- 2.2 The Terminal and Shell
- 2.3 Understanding the Shell Prompt
- 2.4 Basic Navigation Commands
- 2.5 File Management Commands
- 2.6 Getting Help

3. Linux File System

- 3.1 The Filesystem Hierarchy Standard
- 3.2 Important Directories
- 3.3 Absolute vs. Relative Paths
- 3.4 File Types
- 3.5 File Permissions
- 3.6 Ownership
- 3.7 Symbolic and Hard Links

4. User and Group Management

- 4.1 Understanding Users and Groups
- 4.2 The /etc/passwd File
- 4.3 The /etc/shadow File
- 4.4 Creating Users
- 4.5 Modifying Users

- 4.6 Deleting Users
- 4.7 Creating and Managing Groups

5. File Permissions and Ownership

- 5.1 Understanding Permission Types
- 5.2 Permission Numeric Representation
- 5.3 Changing Permissions with chmod
- 5.4 Changing Ownership with chown
- 5.5 Special Permissions
- 5.6 Default Permissions with umask

6. Working with Text Files

- 6.1 Viewing Files
- 6.2 The grep Command
- 6.3 The sed Command
- 6.4 The awk Command
- 6.5 The cut Command
- 6.6 The sort Command
- 6.7 The uniq Command
- 6.8 Pipes and Redirects

7. Package Management

- 7.1 Package Management Overview
- 7.2 APT (Debian/Ubuntu)
- 7.3 YUM/DNF (RHEL/CentOS/Fedora)
- 7.4 Installing Software
- 7.5 Removing Software
- 7.6 Updating Software
- 7.7 Managing Repositories

8. System Services and Processes

- 8.1 Understanding Processes
- 8.2 Viewing Processes with ps
- 8.3 The top and htop Commands
- 8.4 Killing Processes

- 8.5 Background and Foreground Processes
- 8.6 System Services with systemd
- 8.7 Managing Services

9. System Monitoring

- 9.1 CPU Monitoring
- 9.2 Memory Monitoring
- 9.3 Disk Monitoring
- 9.4 Network Monitoring
- 9.5 System Load
- 9.6 Monitoring Tools Summary

10. Disk Management

- 10.1 Understanding Disk Partitions
- 10.2 Viewing Disk Usage
- 10.3 Partitioning with fdisk
- 10.4 Formatting Filesystems
- 10.5 Mounting and Unmounting
- 10.6 Logical Volume Management
- 10.7 Disk Quotas

11. Networking Basics

- 11.1 Network Configuration Files
- 11.2 The ip Command
- 11.3 Network Interface Configuration
- 11.4 DNS Configuration
- 11.5 Routing
- 11.6 Network Troubleshooting

12. Firewall Management

- 12.1 Understanding Firewalls
- 12.2 iptables Basics
- 12.3 Firewalld (RHEL/CentOS)
- 12.4 UFW (Ubuntu/Debian)
- 12.5 Common Firewall Rules

13. SSH and Remote Access

- 13.1 What is SSH?
- 13.2 SSH Configuration
- 13.3 Generating SSH Keys
- 13.4 Copying SSH Keys
- 13.5 SSH Client Usage
- 13.6 SSH Tunneling
- 13.7 Securing SSH

14. System Logs

- 14.1 Understanding System Logs
- 14.2 The /var/log Directory
- 14.3 Journald and systemd Journal
- 14.4 Viewing Logs with journalctl
- 14.5 Log Rotation
- 14.6 Analyzing Logs

15. Backup and Recovery

- 15.1 Backup Strategies
- 15.2 The tar Command
- 15.3 The rsync Command
- 15.4 Automating Backups
- 15.5 Disaster Recovery

16. Cron and Scheduled Tasks

- 16.1 What is Cron?
- 16.2 Cron Syntax
- 16.3 Managing Cron Jobs
- 16.4 User-Specific Cron
- 16.5 System-Wide Cron
- 16.6 Anacron and Systemd Timers

17. Shell Scripting

- 17.1 What is Shell Scripting?

- 17.2 Creating a Script
- 17.3 Variables
- 17.4 Conditional Statements
- 17.5 Loops
- 17.6 Functions
- 17.7 Script Best Practices

18. Networking Services

- 18.1 DNS Server Basics
- 18.2 NTP Time Synchronization
- 18.3 Web Server (Apache/Nginx)
- 18.4 Database Server (MySQL/PostgreSQL)
- 18.5 DHCP Server
- 18.6 File Server (NFS/Samba)

19. Security Hardening

- 19.1 Security Principles
- 19.2 System Updates
- 19.3 Disabling Unused Services
- 19.4 Security-Enhanced Linux
- 19.5 AppArmor
- 19.6 User Security
- 19.7 File System Security

20. Performance Tuning

- 20.1 Performance Metrics
- 20.2 CPU Tuning
- 20.3 Memory Tuning
- 20.4 Disk I/O Tuning
- 20.5 Network Tuning
- 20.6 Kernel Tuning

21. Troubleshooting

- 21.1 Troubleshooting Methodology
- 21.2 Boot Issues

- 21.3 Application Issues
- 21.4 Network Issues
- 21.5 Permission Issues
- 21.6 Resource Issues

22. Virtualization

- 22.1 Virtualization Overview
- 22.2 KVM Virtualization
- 22.3 Managing Virtual Machines
- 22.4 Docker Basics
- 22.5 Container Management

23. System Updates and Upgrades

- 23.1 Update Strategy
- 23.2 Security Updates
- 23.3 Upgrading Packages
- 23.4 Upgrading Distributions
- 23.5 Managing Updates with Unattended-Upgrades

24. Conclusion

Absolute paths start from the root directory (`/`) and specify the full location (e.g., `/home/john/file.txt`). Relative paths start from the current directory (e.g., `documents/file.txt` or `../file.txt`). `.` (single dot) refers to the current directory, `..` refers to the parent.

Example: If you're in `/home/john` , `documents/file.txt` is relative and points to `/home/john/documents/file.txt` . `/tmp/file.txt` is absolute and points to `/tmp/file.txt` regardless of your current location.

3.4 File Types

Linux supports several file types: regular files (`-`), directories (`d`), symbolic links (`l`), device files (`b` for block, `c` for character), socket files (`s`), and FIFO/pipe files (`p`). The first character of `ls -l` output shows the file type.

Example: `ls -l` shows `-rw-r--r--` (regular file), `drwxr-xr-x` (directory), `lrwxrwxrwx` (symbolic link), `brw-rw----` (block device), `crw-rw----` (character device).

3.5 File Permissions

Linux uses a permission system with three types of access: read (`r`), write (`w`), and execute (`x`). These are granted to three user categories: owner (user), group, and others. Permissions are displayed as a 10-character string in `ls -l` output.

Example: `-rwxr-xr--` indicates owner has read/write/execute, group has read/execute, others have read only. The first character indicates file type (`-` for regular file).

3.6 Ownership

Every file and directory has an owner (user) and a group. The owner is usually the creator of the file. The group determines the permissions for users in that group. `ls -l` displays ownership as user:group.

Example: `-rw-r--r-- 1 john developers 1024 Jan 1 12:00 file.txt` means the owner is john, the group is developers. john has read/write permissions, developers have read-only, others have read-only.

3.7 Symbolic and Hard Links

Symbolic links (symlinks) are special files that point to other files or directories. They're created with `ln -s` and can link to targets on different filesystems. Hard links are direct references to the same inode (file data). They cannot span filesystems.

The Linux philosophy emphasizes simplicity, modularity, and reusability. Key principles include: everything is a file (hardware, processes, and data are represented as files), small, single-purpose programs work together, configuration is stored in plain text files, and command-line utilities can be combined with pipes for powerful operations.

Example: The `grep` command searches text, and `wc` counts lines. Combining them with a pipe `grep error logfile | wc -l` counts error lines in a log file. Each tool does one thing well, and they work together.

1.5 Roles and Responsibilities

A Linux administrator's responsibilities include: installing and configuring Linux servers, managing user accounts and permissions, performing system updates and patching, monitoring system performance and health, implementing security measures, managing storage and backups, troubleshooting issues, and automating routine tasks through scripting.

Example: A typical day includes monitoring dashboard alerts, applying critical security patches, investigating a reported permission issue, creating a new user account for a new employee, and writing a script to automate daily backup verification.

2. Getting Started with Linux

2.1 Accessing a Linux System

You can access a Linux system in several ways: directly on a physical machine (server or desktop), via SSH (remote access), through a cloud instance (AWS EC2, Azure VM), or using virtual machines (VirtualBox, VMware). For learning, virtual machines or cloud instances are recommended because they allow experimentation without risk.

Example: You create an Ubuntu VM on VirtualBox, install it, and start the system. You see the login prompt. After logging in, you can start using the command line. Alternatively, you launch an AWS EC2 instance and SSH into it.

2.2 The Terminal and Shell

The terminal is a window that allows you to interact with the operating system through text commands. The shell is the program that interprets these commands. Common shells include Bash (most common), Zsh, and Fish. The default shell on most Linux distributions is Bash (Bourne Again SHell).

Example: On Ubuntu, you open the Terminal application to start a shell session. You see a prompt `user@host:~$`. Type `echo "Hello"` and press

Enter. The shell runs the `echo` command, outputs "Hello."

2.3 Understanding the Shell Prompt

The shell prompt is the line where you type commands. It typically shows:

`username@hostname:current_directory$` (for regular users) or
`root@hostname:current_directory#` (for root users). `$` indicates a regular user, `#` indicates root user. `~` represents your home directory.

Example: `john@webserver:~$` means user "john" on host "webserver," currently in the home directory (`~`). `root@webserver:/etc#` means root user, in the `/etc` directory.

2.4 Basic Navigation Commands

`pwd` (print working directory) shows your current directory. `ls` lists files and directories in the current location. `cd` changes the current directory. `cd ..` moves up one level. `cd ~` or `cd` goes to your home directory. `cd /` goes to the root directory.

Example:

```
# /home/john
pwd
# file1.txt documents
ls
# Move to documents directory
cd documents
# /home/john/documents
pwd
# /home/john
cd ..
```

2.5 File Management Commands

`touch` creates an empty file or updates file timestamp. `cp` copies files and directories. `mv` moves or renames files/directories. `rm` removes files (use `-r` for directories). `mkdir` creates directories. `rmdir` removes empty directories.

Example:

```
# Create empty file
touch myfile.txt
# Copy file
cp myfile.txt backup.txt
# Rename file
mv backup.txt archive.txt
```

```
# Create directory
mkdir myfolder
# Remove file
rm myfile.txt
```

2.6 Getting Help

`man` (manual) displays the manual pages for a command. `command --help` shows brief usage information. `apropos` searches manual pages. The `info` command displays GNU documentation. Use `man -k` to search manual page summaries.

Example:

```
# Manual for ls command
man ls
# Brief help for ls
ls --help
# Find commands related to copy
apropos copy
```

3. Linux File System

3.1 The Filesystem Hierarchy Standard

The Filesystem Hierarchy Standard (FHS) defines the directory structure and content for Linux systems. It ensures consistency across distributions. The root directory (`/`) is the top-level directory. All other directories branch from root. The FHS is designed to create a predictable and organized filesystem.

Example: Regardless of whether you're using Ubuntu, CentOS, or Debian, the system looks for essential binaries in `/bin`, configuration files in `/etc`, and logs in `/var/log`.

3.2 Important Directories

Key directories in the FHS: `/bin` and `/usr/bin` (essential user binaries), `/sbin` (system binaries), `/etc` (system configuration), `/home` (user home directories), `/var` (variable data like logs), `/tmp` (temporary files), `/dev` (device files), `/proc` (process information), and `/boot` (boot loader files).

Example: Apache configuration is at `/etc/apache2` or `/etc/httpd`. Application logs are in `/var/log`. User-specific files are in `/home/username`.

3.3 Absolute vs. Relative Paths

Example: `ln -s /etc/nginx/nginx.conf /tmp/nginx.conf` creates a symbolic link. Hard link: `ln /etc/nginx/nginx.conf /tmp/nginx-hard.conf`. Both point to the same data, but the hard link must be on the same filesystem.

4. User and Group Management

4.1 Understanding Users and Groups

A user account is an identity that allows someone to log into a system. Each user has a unique User ID (UID) and belongs to one or more groups. Groups are collections of users with shared permissions. The primary group is assigned to files created by the user. Secondary groups provide additional permissions.

Example: User "jane" has UID 1001 and belongs to the primary group "developers" and secondary group "webadmin." Files created by jane belong to "developers." The "webadmin" group gives jane access to web server configuration.

4.2 The /etc/passwd File

`/etc/passwd` stores user account information. Each line has seven fields: username, password placeholder (x), UID, primary GID, GECOS (real name), home directory, and login shell. The actual passwords are stored in `/etc/shadow` for security.

Example: `john:x:1001:1001:John Doe:/home/john:/bin/bash` means user john, UID 1001, GID 1001, real name John Doe, home at `/home/john`, bash shell.

4.3 The /etc/shadow File

`/etc/shadow` contains encrypted user passwords and password aging information. Fields include: username, encrypted password, last change date, min days between changes, max days valid, warning period, inactivity period, and expiration date. This file is only readable by root.

Example: `john:$6$salt$encryptedpassword:19000:0:99999:7:::` (encrypted password, last changed on day 19000, no minimum days, 99999 days validity, 7-day warning).

4.4 Creating Users

`useradd` creates a new user account. Options include `-m` (create home directory), `-s` (set shell), `-G` (secondary groups), and `-u` (specify UID).

`adduser` is an interactive wrapper for `useradd` on Debian systems.

Example:

```
useradd -m -s /bin/bash -G sudo,developers john
```

This creates user `john`, creates home directory `/home/john`, sets login shell to `bash`, and adds `john` to `sudo` and `developers` groups.

4.5 Modifying Users

`usermod` modifies existing user accounts. Common options: `-G` (add to groups), `-aG` (append to groups), `-s` (change shell), `-L` (lock account), `-U` (unlock account). `passwd` changes user passwords.

Example:

```
# Add john to docker group
usermod -aG docker john
# Change shell to zsh
usermod -s /bin/zsh john
# Change john's password
passwd john
```

4.6 Deleting Users

`userdel` removes user accounts. Use `-r` to also remove the user's home directory and mail spool. Warning: deleting a user permanently removes their data.

Example:

```
# Remove user only
userdel john
# Remove user and home directory
userdel -r john
```

4.7 Creating and Managing Groups

`groupadd` creates a new group. `groupmod` modifies groups. `groupdel` deletes groups. `gpasswd` manages group passwords. Users can be added to groups with `usermod -aG` or `gpasswd -a`.

Example:

```
groupadd developers
gpasswd -a john developers
# Rename group developers to devs
```

```
groupmod -n devs developers
# Delete group devs
groupdel devs
```

5. File Permissions and Ownership

5.1 Understanding Permission Types

The three permission types are: read (r) - allows viewing file contents or listing directory contents; write (w) - allows modifying files or creating/deleting files in directories; execute (x) - allows running executable files or entering directories. These are assigned to owner, group, and others.

Example: For a script file `script.sh`, `-rwxr-xr-x` means owner can read/write/execute, group can read/execute, others can read/execute. For a directory, `drwxr-xr-x` means owner can list, create, and delete files; others can only list and enter.

5.2 Permission Numeric Representation

Permissions can be represented numerically: read=4, write=2, execute=1. The total is the sum of permissions. For owner, group, others, three numbers represent the permissions (e.g., 755 = 7 for owner, 5 for group, 5 for others). This is known as octal notation.

Example: `chmod 755 file` sets owner: read+write+execute (7), group: read+execute (5), others: read+execute (5). `chmod 644` sets owner: read+write (6), group: read (4), others: read (4).

5.3 Changing Permissions with chmod

`chmod` changes file permissions. Use symbolic mode (`u` for user, `g` for group, `o` for others, `a` for all) with `+`, `-`, `=` operators. Or use numeric (octal) mode. The `-R` flag applies permissions recursively to directories.

Example:

```
# Add execute permission for owner
chmod u+x script.sh
# Remove write permission for group
chmod g-w file.txt
# Set rwxr-x--- permissions
chmod 750 script.sh
# Recursive permissions
chmod -R 755 /var/www
```

5.4 Changing Ownership with chown

`chown` changes file owner and group. Use `chown user:group file` to change both, `chown user file` to change owner only, `chown :group file` to change group only. Use `-R` for recursive changes.

Example:

```
chown john:developers file.txt
chown john file.txt
chown :developers file.txt
chown -R www-data:www-data /var/www
```

5.5 Special Permissions

Special permissions add functionality beyond regular read/write/execute: SetUID (SUID) - executed file runs as owner; SetGID (SGID) - executed file runs as group or directory creation inherits group; Sticky Bit - only file owner can delete files in the directory.

Example: `passwd` command has SUID so users can change their passwords. `/tmp` has sticky bit so users can only delete their own files. SUID is set with `chmod u+s`, SGID with `chmod g+s`, sticky with `chmod +t`.

5.6 Default Permissions with umask

`umask` sets default permissions for newly created files and directories. It removes (masks) permissions. The default umask is often 022 (removes write for group and others). Directories get 777 minus umask; files get 666 minus umask.

Example: With umask 022, new directories get 755, new files get 644. With umask 002, new directories get 775, new files get 664. Set umask in `/etc/profile` or `~/.profile`.

6. Working with Text Files

6.1 Viewing Files

`cat` displays the entire file content. `less` displays content page by page (more powerful than `more`). `head` shows the first lines (default 10). `tail` shows the last lines (default 10). `tail -f` follows a file in real-time (useful for logs).

Example:

```
cat /etc/passwd
less /var/log/syslog
head -20 access.log
tail -f /var/log/nginx/error.log
```

6.2 The grep Command

grep searches for patterns in files or input. It uses regular expressions. Common options: **-i** (ignore case), **-v** (invert match), **-r** (recursive), **-l** (list filenames only), and **-n** (show line numbers).

Example:

```
grep "error" /var/log/nginx/error.log
grep -i "warning" logfile
grep -r "ServerName" /etc/nginx/
grep -l "error" *.log
grep -v "INFO" app.log
```

6.3 The sed Command

sed (stream editor) performs text transformations on a stream of text. Common uses: search and replace (**s/old/new/g**), deleting lines (**d**), and printing lines (**p**). **sed** is powerful for automated text processing.

Example:

```
# Replace all foo with bar
sed 's/foo/bar/g' file.txt
# In-place replacement
sed -i 's/old/new/g' file.txt
# Delete lines with pattern
sed '/pattern/d' file.txt
# Print lines with error
sed -n '/error/p' logfile
```

6.4 The awk Command

awk is a pattern scanning and processing language. It's excellent for structured data like CSV or log files. It splits lines into fields (accessed as **\$1**, **\$2**, etc.) and applies actions based on patterns.

Example:

```
# Print first and third fields
awk '{print $1, $3}' logfile
# Print lines with error
awk '/error/ {print $0}' logfile
```

```
# Sum second column
awk '{sum+=$2} END {print sum}' data
# Print usernames
awk -F: '{print $1}' /etc/passwd
```

6.5 The cut Command

`cut` extracts sections from each line of input. It works with delimiters (`-d`), fields (`-f`), or characters (`-c`). It's simpler than `awk` for basic column extraction.

Example:

```
# First field of colon-separated file
cut -d: -f1 /etc/passwd
# Fields 2 through 4 with space delimiter
cut -d' ' -f2-4 data.txt
# First five characters of each line
cut -c1-5 file.txt
```

6.6 The sort Command

`sort` sorts lines of text. Options: `-n` (numeric sort), `-r` (reverse), `-k` (key field), `-t` (field delimiter), and `-u` (unique). It's often used with pipes.

Example:

```
# Alphabetical sort
sort file.txt
# Numeric sort
sort -n numbers.txt
# Sort by second colon-separated field
sort -k2 -t: data.txt
# Reverse unique sort
sort -ru file.txt
```

6.7 The uniq Command

`uniq` reports or omits repeated lines. It only works on adjacent duplicate lines, so it's often used after `sort`. Options include `-c` (count occurrences), `-d` (only duplicate lines), and `-u` (only unique lines).

Example:

```
# Remove duplicate lines
sort logfile | uniq
# Count occurrences
sort logfile | uniq -c
```

```
# Show duplicates only
sort logfile | uniq -d
# Show unique lines only
sort logfile | uniq -u
```

6.8 Pipes and Redirects

Pipes (`|`) send output of one command as input to another. Redirects: `>` (overwrite), `>>` (append), `<` (input from file), `2>` (redirect stderr), `2>&1` (redirect stderr to stdout). This enables powerful command chaining.

Example:

```
grep error logfile | wc -l
cat file1 file2 > combined
echo "new line" >> logfile
cat < input.txt
find / -name "*.conf" 2>/dev/null
```

7. Package Management

7.1 Package Management Overview

Package management is the system for installing, updating, and removing software on Linux. Different distributions use different package managers: Debian/Ubuntu use APT (Advanced Package Tool), RHEL/CentOS/Fedora use YUM/DNF, and openSUSE uses Zypper. Packages contain compiled binaries, configuration files, and metadata.

Example: To install a web server on Ubuntu, you use `sudo apt install apache2`. On CentOS, you use `sudo yum install httpd`. Both achieve the same result using different package managers.

7.2 APT (Debian/Ubuntu)

APT is the package manager for Debian-based systems. Key commands: `apt update` (refresh package list), `apt upgrade` (upgrade packages), `apt install` (install), `apt remove` (remove), `apt purge` (remove with config), `apt search` (search), and `apt show` (package details).

Example:

```
# Update package list
sudo apt update
# Upgrade all packages
sudo apt upgrade
# Install Nginx
```

```
sudo apt install nginx
# Remove Nginx
sudo apt remove nginx
# Remove with config
sudo apt purge nginx
# Search for Python packages
apt search python
```

| 7.3 YUM/DNF (RHEL/CentOS/Fedora)

YUM (Yellowdog Updater Modified) and its successor DNF are package managers for RPM-based systems. Key commands: `dnf makecache` (refresh cache), `dnf install` (install), `dnf remove` (remove), `dnf update` (update), `dnf search` (search), and `dnf info` (package info).

Example:

```
# Refresh cache
sudo dnf makecache
# Update all packages
sudo dnf update
# Install Nginx
sudo dnf install nginx
# Remove Nginx
sudo dnf remove nginx
# Search for Python packages
dnf search python
```

| 7.4 Installing Software

Installing software involves the package manager resolving dependencies and downloading packages from repositories. Use `apt install` (Debian) or `dnf install` (RHEL). You can install specific versions or packages from local files.

Example:

```
# Debian
# Install without confirmation
sudo apt install -y curl
# Install local .deb file
sudo apt install ./package.deb

# RHEL
# Install without confirmation
sudo dnf install -y curl
```

```
# Install local .rpm file
sudo dnf install ./package.rpm
```

7.5 Removing Software

Removing software removes the package but may leave configuration files. Use `apt remove` (Debian) or `dnf remove` (RHEL). `purge` removes configuration files as well. `autoremove` removes unneeded dependencies.

Example:

```
# Debian
# Remove package
sudo apt remove nginx
# Remove package and config
sudo apt purge nginx
# Remove unneeded dependencies
sudo apt autoremove

# RHEL
# Remove package
sudo dnf remove nginx
# Remove unneeded dependencies
sudo dnf autoremove
```

7.6 Updating Software

Regular updates are crucial for security. `apt update` refreshes package metadata. `apt upgrade` installs updates. `apt dist-upgrade` handles major upgrades. For RHEL, `dnf update` refreshes and upgrades. Security updates are often prioritized.

Example:

```
# Debian
sudo apt update
# Install updates
sudo apt upgrade
# Distribution upgrade
sudo apt dist-upgrade

# RHEL
# Update all packages
sudo dnf update
```

7.7 Managing Repositories

Repositories are sources of packages. On Debian, repositories are in `/etc/apt/sources.list` and `/etc/apt/sources.list.d/`. Add repositories with `add-apt-repository`. On RHEL, repositories are managed with `.repo` files in `/etc/yum.repos.d/`.

Example:

```
# Debian
sudo add-apt-repository ppa:nginx/stable
sudo apt update

# RHEL
sudo dnf config-manager --add-repo https://repo.nginx.com/nginx.repo
```

8. System Services and Processes

8.1 Understanding Processes

A process is a running instance of a program. Each process has a Process ID (PID). Processes can be in various states: running, sleeping, stopped, or zombie. The parent process (PPID) is the process that started it. Processes are managed by the kernel.

Example: When you run `ls`, the shell creates a process with a unique PID. When the command completes, the process terminates. The shell (your terminal) is also a process with its own PID.

8.2 Viewing Processes with `ps`

`ps` (process status) displays information about running processes. Common options: `ps aux` (all processes, user-oriented format), `ps ef` (full-format listing), and `ps -ef --forest` (tree view). `ps aux` is the most commonly used.

Example:

```
# All processes with details
ps aux
# Full format listing
ps -ef
# Find nginx processes
ps aux | grep nginx
# Processes owned by john
ps -u john
```

8.3 The top and htop Commands

`top` provides a dynamic, real-time view of processes. It shows CPU usage, memory usage, load average, and updates every few seconds. `htop` is an improved version with color, mouse support, and easier process management. Install `htop` with `sudo apt install htop` or `sudo dnf install htop`.

Example: Run `top` to see real-time process information. Press `P` to sort by CPU, `M` to sort by memory, `k` to kill a process, `q` to quit. Run `htop` for an interactive, user-friendly interface.

8.4 Killing Processes

`kill` sends signals to processes. The most common signal is SIGTERM (`kill PID`) which allows graceful termination. SIGKILL (`kill -9 PID`) forcefully terminates. `pkill` kills processes by name. `killall` kills all processes with a given name.

Example:

```
# Terminate PID 1234 gracefully
kill 1234
# Force terminate PID 1234
kill -9 1234
# Terminate all nginx processes
pkill nginx
# Terminate all node processes
killall node
```

8.5 Background and Foreground Processes

Commands run in the foreground by default. Use `&` to run in the background. `jobs` lists background jobs. `fg` brings a job to the foreground. `bg` resumes a suspended job in the background. Use Ctrl+Z to suspend a foreground job.

Example:

```
# Start in background
./long_running.sh &
# List background jobs
jobs
# Bring job 1 to foreground
fg %1
# Resume job 2 in background
bg %2
```

8.6 System Services with systemd

Systemd is the init system and service manager on most modern Linux distributions. It replaces SysV init. Services are managed by systemd units, with service files in `/etc/systemd/system/` and `/lib/systemd/system/`.

Example: `systemctl` is the primary command for managing services. Services are defined with `.service` files. Each service specifies how to start, stop, and restart.

8.7 Managing Services

`systemctl` commands manage services: `start`, `stop`, `restart`, `status`, `enable` (start at boot), `disable` (remove from boot), `mask` (prevent starting), and `unmask`. `systemctl list-units --type=service` lists all services.

Example:

```
# Start nginx service
sudo systemctl start nginx
# Stop nginx service
sudo systemctl stop nginx
# Restart nginx
sudo systemctl restart nginx
# Check status
sudo systemctl status nginx
# Start at boot
sudo systemctl enable nginx
# Don't start at boot
sudo systemctl disable nginx
```

9. System Monitoring

9.1 CPU Monitoring

CPU utilization is measured as the percentage of time the CPU is busy. Monitoring tools: `top` (real-time CPU usage), `mpstat` (per-CPU stats), `sar` (historical stats), `uptime` (load average), and `vmstat` (system-wide stats). High CPU usage can indicate a performance issue.

Example:

```
# Show processes by user
top -u nginx
# All CPUs every 5 seconds
mpstat -P ALL 5
```

```
# Virtual memory stats every 2s
vmstat 2
# Load average
uptime
```

9.2 Memory Monitoring

Memory monitoring includes RAM, swap, and cache usage. Tools include `free` (simple memory summary), `vmstat` (virtual memory stats), `top` (process memory usage), and `smem` (more accurate memory reporting). High memory usage can cause swapping and performance degradation.

Example:

```
# Human-readable memory summary
free -h
# Memory statistics
vmstat -s
# Sort top by memory
top -o %MEM
```

9.3 Disk Monitoring

Disk monitoring tracks usage, I/O, and performance. `df` (disk free) shows filesystem usage. `du` (disk usage) shows directory sizes. `iostat` shows I/O statistics. `iostat` shows per-process I/O usage.

Example:

```
# Filesystem usage
df -h
# Size of /var/log
du -sh /var/log
# Extended I/O stats every 2s
iostat -x 2
# Per-process I/O stats
iotop
```

9.4 Network Monitoring

Network monitoring tracks interface usage, bandwidth, connections, and errors. `ifconfig` or `ip addr` (interface info), `ss` (socket statistics), `netstat` (network statistics), `iftop` (bandwidth usage), and `nethogs` (per-process bandwidth).

Example:

```
# Listening ports
ss -tulpn
# Bandwidth on eth0
iftop -i eth0
# Per-process bandwidth
nethogs
```

9.5 System Load

Load average shows the average number of processes in the run queue over 1, 5, and 15 minutes. Values > 1 (per CPU core) indicate the system is overloaded. Check with `uptime`, `top`, or `htop`.

Example: `uptime` shows: 15:30:23 up 14 days, 2:23, load average: 1.23, 0.89, 0.56. A load of 1.23 on a 4-core system means average 0.3 processes per core, which is fine. On a single-core system, it's too high.

9.6 Monitoring Tools Summary

Several tools provide comprehensive monitoring: `nmon` (all-in-one), `glances` (system monitoring tool), `sar` (system activity reporter), and `atop` (detailed process monitoring). Choose based on your needs.

Example: `sar -u 2 5` collects CPU usage 5 times every 2 seconds. `glances` provides a single-screen dashboard.

10. Disk Management

10.1 Understanding Disk Partitions

Disk partitions divide a physical disk into separate areas. Partitions are represented as device files: `/dev/sda1`, `/dev/sda2`, etc. Partitions are created with tools like `fdisk`, `gdisk`, or `parted`. Common partition tables: MBR (Master Boot Record) and GPT (GUID Partition Table).

Example: A server with a single physical disk `/dev/sda` has partitions: `/dev/sda1` (boot), `/dev/sda2` (root), `/dev/sda3` (swap). Each partition is formatted with a filesystem.

10.2 Viewing Disk Usage

`df` shows filesystem usage. `du` shows directory sizes. `lsblk` shows disk layout. `parted -l` shows partition table. These commands help identify which directories are consuming space.

Example:

```
# Human-readable filesystem usage
df -h
# Size of /var
du -sh /var
# Size of each user's home directory
du -ch /home/* | sort -hr
# Show disk layout
lsblk
```

10.3 Partitioning with fdisk

`fdisk` is a command-line partition editor. Common subcommands: `p` (print table), `n` (new partition), `d` (delete partition), `w` (write changes), `q` (quit without saving). For disks larger than 2TB, use `gdisk` or `parted`.

Example:

```
# Start fdisk for /dev/sdb
sudo fdisk /dev/sdb
# (Interactive session)
# n - create new partition
# p - primary
# 1 - partition number
# [Enter] - default first sector
# [Enter] - default last sector
# w - write changes and exit
```

10.4 Formatting Filesystems

After creating partitions, format them with `mkfs` (make filesystem). Common filesystems: `ext4` (`mkfs.ext4`), `xfs` (`mkfs.xfs`), `btrfs` (`mkfs.btrfs`). Choose based on your needs: `ext4` for general use, `xfs` for large files, `btrfs` for advanced features.

Example:

```
# Format as ext4
sudo mkfs.ext4 /dev/sdb1
# Format as xfs
sudo mkfs.xfs /dev/sdc1
# Format as btrfs
sudo mkfs.btrfs /dev/sdd1
```

10.5 Mounting and Unmounting

Mounting attaches a filesystem to a directory. `mount` mounts filesystems, `umount` unmounts. Mounts can be temporary (`mount /dev/sdb1`

`/mnt/data`) or permanent (in `/etc/fstab`). The `findmnt` command shows mount points.

Example:

```
# Mount partition
sudo mount /dev/sdb1 /mnt/data
# Unmount partition
sudo umount /mnt/data
echo '/dev/sdb1 /mnt/data ext4 defaults 0 0' | sudo tee -a /etc/fstab
```

10.6 Logical Volume Management

LVM (Logical Volume Management) provides flexible storage management. Key concepts: Physical Volumes (PV), Volume Groups (VG), and Logical Volumes (LV). LVM allows resizing, snapshots, and striping.

Example:

```
# Create PV
sudo pvcreate /dev/sdb1
# Create VG
sudo vgcreate vg_data /dev/sdb1
# Create LV
sudo lvcreate -L 10G -n lv_data vg_data
sudo mkfs.ext4 /dev/vg_data/lv_data
sudo mount /dev/vg_data/lv_data /mnt/data
```

10.7 Disk Quotas

Disk quotas limit user or group disk usage. They prevent any single user from consuming all disk space. Quotas are enabled per filesystem with the `usrquota` and `grpquota` mount options.

Example:

```
# Add usrquota to /etc/fstab
# /dev/vg_data/lv_data /home ext4 defaults,usrquota 0 0

# Remount with quota
sudo mount -o remount /home
# Create quota database
sudo quotacheck -cu /home
# Enable quotas
sudo quotaon /home
```

```
# Set quota for user john
sudo edquota john
```

11. Networking Basics

11.1 Network Configuration Files

Network configuration is stored in files. On Debian/Ubuntu:

`/etc/network/interfaces` or `/etc/netplan/*.yaml`. On RHEL/CentOS: `/etc/sysconfig/network-scripts/ifcfg-eth0`. Modern systems may use `NetworkManager` for configuration.

Example: Debian/Ubuntu `interfaces` file: `auto eth0, iface eth0 inet static, address 192.168.1.10, netmask 255.255.255.0, gateway 192.168.1.1`.

11.2 The ip Command

The `ip` command is the modern tool for network configuration.

Subcommands: `ip addr` (show interfaces), `ip link` (manage interfaces), `ip route` (manage routing), and `ip neigh` (ARP table). It replaces older commands like `ifconfig` and `route`.

Example:

```
# Show all interfaces
ip addr show
# Bring eth0 up
ip link set eth0 up
# Add IP address
ip addr add 192.168.1.10/24 dev eth0
# Add default route
ip route add default via 192.168.1.1
```

11.3 Network Interface Configuration

Network interfaces can be configured statically or with DHCP. Static configuration requires IP address, netmask, gateway, and DNS servers. DHCP configuration requests these from a DHCP server.

Example: Static configuration in `netplan` (Ubuntu 18+) with `/etc/netplan/01-netcfg.yaml`:

```
network:
  version: 2
  ethernets:
    eth0:
```

```
dhcp4: false
addresses: [192.168.1.10/24]
routes:
  - to: default
    via: 192.168.1.1
nameservers:
  addresses: [8.8.8.8, 8.8.4.4]
```

11.4 DNS Configuration

DNS resolution is configured in `/etc/resolv.conf`. It contains `nameserver` entries and search domains. Modern systems may use `systemd-resolved` or `NetworkManager` which override `/etc/resolv.conf`. Check with `resolvectl`.

Example: `/etc/resolv.conf`:

```
nameserver 8.8.8.8
nameserver 8.8.4.4
search example.com
```

11.5 Routing

Routing determines how network traffic is forwarded. The default route (gateway) handles traffic to other networks. Static routes can be added for specific destinations. Use `ip route` to view and manage routes.

Example:

```
# Show routing table
ip route show
# Add static route
ip route add 10.0.0.0/24 via 192.168.1.2
# Delete route
ip route del 10.0.0.0/24
```

11.6 Network Troubleshooting

Network troubleshooting tools: `ping` (check connectivity), `traceroute` (path to destination), `nslookup` / `dig` (DNS lookup), `curl` (test web service), and `netstat` / `ss` (socket information). Use these to diagnose network issues.

Example:

```
# Check connectivity
ping 8.8.8.8
# Trace path
```

```
tracert google.com
# DNS lookup
nslookup example.com
# Detailed DNS query
dig google.com
# Check HTTP headers
curl -I https://example.com
# List listening ports
ss -tulpn
```

12. Firewall Management

12.1 Understanding Firewalls

A firewall controls incoming and outgoing network traffic based on security rules. Linux firewalls use netfilter (kernel-level packet filtering). Common frontends: iptables (legacy), nftables (modern), firewalld (RHEL/CentOS), and UFW (Ubuntu). Firewalls are essential for security.

Example: You have a web server running on port 80. The firewall allows incoming traffic on port 80 but blocks all other ports. This prevents attackers from accessing services like SSH (port 22) that aren't intended for public access.

12.2 iptables Basics

`iptables` is the traditional firewall tool. It operates on tables (filter, nat, mangle) and chains (INPUT, OUTPUT, FORWARD). Rules specify conditions and actions (ACCEPT, DROP, REJECT). `iptables-save` saves rules, `iptables-restore` loads them.

Example:

```
# List rules with details
sudo iptables -L -v -n
# Allow HTTP
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
# Drop all other traffic
sudo iptables -A INPUT -j DROP
# Save rules
sudo iptables-save > /etc/iptables.rules
```

12.3 Firewalld (RHEL/CentOS)

`firewalld` is a dynamic firewall manager for RHEL/CentOS. It manages zones (public, internal, trusted) and services (http, ssh, mysql). Changes are

applied immediately. Commands: `firewall-cmd` (manage), `--add-service`, `--add-port`, `--permanent` (make persistent).

Example:

```
# Show current rules
sudo firewall-cmd --list-all
# Allow HTTP
sudo firewall-cmd --add-service=http --permanent
# Allow custom port
sudo firewall-cmd --add-port=8080/tcp --permanent
# Reload to apply permanent rules
sudo firewall-cmd --reload
```

12.4 UFW (Ubuntu/Debian)

UFW (Uncomplicated Firewall) is a user-friendly interface for iptables on Ubuntu/Debian. Commands: `ufw enable/disable` (toggle), `ufw allow` (allow port/service), `ufw deny` (deny), and `ufw status` (show rules). It simplifies firewall management.

Example:

```
# Enable firewall
sudo ufw enable
# Allow SSH
sudo ufw allow ssh
# Allow HTTP
sudo ufw allow 80/tcp
# Allow HTTPS
sudo ufw allow 443/tcp
# Allow from local network
sudo ufw allow from 192.168.1.0/24
# Show rules with numbers
sudo ufw status numbered
# Delete rule number 2
sudo ufw delete 2
```

12.5 Common Firewall Rules

Common rules: allow SSH (port 22) from trusted IPs, allow HTTP/HTTPS (80/443) from anywhere, allow MySQL (3306) only from the application server, allow DNS (53), and block everything else. Default policy should be DROP.

Example:

```
# iptables
# Default deny
sudo iptables -P INPUT DROP
# SSH
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
# HTTP
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -
j ACCEPT

# UFW
# Deny all incoming
sudo ufw default deny incoming
# Allow all outgoing
sudo ufw default allow outgoing
# SSH only from specific IP
sudo ufw allow from 192.168.1.100 to any port 22
```

13. SSH and Remote Access

13.1 What is SSH?

SSH (Secure Shell) is a protocol for secure remote access to systems. It encrypts all communication, replacing insecure protocols like Telnet and FTP. SSH provides remote shell access, file transfer, and port forwarding. The server is `sshd`, the client is `ssh`.

Example: You're working from home and need to access a server in the office. You use SSH to securely log into the server, run commands, and transfer files. The connection is encrypted, preventing eavesdropping.

13.2 SSH Configuration

The SSH server configuration is `/etc/ssh/sshd_config`. Key settings: `Port` (change from 22 for security), `PermitRootLogin` (disable root login), `PasswordAuthentication` (disable for key-based auth), and `AllowUsers` (specify allowed users). Reload after changes.

Example: `/etc/ssh/sshd_config`:

```
# Change SSH port
Port 2222
# Disable root login
PermitRootLogin no
# Disable password auth
PasswordAuthentication no
```

```
# Only allow specific users
AllowUsers john admin
```

13.3 Generating SSH Keys

SSH key pairs are the most secure authentication method. Generate a key pair with `ssh-keygen`. The default is RSA 3072-bit (or Ed25519 for better security). The private key stays on your client, the public key is copied to servers.

Example:

```
# Generate Ed25519 key
ssh-keygen -t ed25519 -C "john@example.com"
# Generate RSA key
ssh-keygen -t rsa -b 4096 -C "john@example.com"
```

13.4 Copying SSH Keys

Copy your public key to a server with `ssh-copy-id`. This adds the public key to the server's `~/.ssh/authorized_keys` file. Alternatively, manually append the public key. After copying, you can log in without a password.

Example:

```
# Copy public key
ssh-copy-id john@server
# Passwordless login
ssh john@server
```

13.5 SSH Client Usage

Basic SSH usage: `ssh user@host`. Options: `-p` (port), `-i` (identity file), `-v` (verbose/debug). `scp` securely copies files. `sftp` is an interactive file transfer tool. SSH config file `~/.ssh/config` simplifies connections.

Example:

```
# Connect on custom port
ssh -p 2222 john@server
# Use specific key
ssh -i ~/.ssh/mykey.pem ubuntu@ec2-instance
# Copy file to server
scp file.txt john@server:/tmp/
# Interactive file transfer
sftp john@server
```

13.6 SSH Tunneling

SSH tunneling forwards ports through an SSH connection. Local port forwarding (`-L`) forwards a local port to a remote host. Remote port forwarding (`-R`) forwards a remote port to a local host. Dynamic port forwarding (`-D`) creates a SOCKS proxy.

Example:

```
# Access server's web on local port 8080
ssh -L 8080:localhost:80 john@server
# Dynamic SOCKS proxy on port 1080
ssh -D 1080 john@server
```

13.7 Securing SSH

Secure SSH by disabling root login, using key-based authentication, changing the default port, using fail2ban (blocks repeated failures), and keeping SSH updated. Monitor `/var/log/auth.log` for failed login attempts.

Example: Install and configure fail2ban for SSH: `sudo apt install fail2ban` . It monitors logs and blocks IPs with too many failed attempts. Configure in `/etc/fail2ban/jail.local` .

14. System Logs

14.1 Understanding System Logs

System logs record events on the system. They're essential for troubleshooting, security auditing, and monitoring. Logs are stored in `/var/log/` . Key log files: `syslog` (general system), `auth.log` (authentication), `kern.log` (kernel), and `dmesg` (boot messages).

Example: When you have a service failing, you check `/var/log/syslog` or `/var/log/nginx/error.log` to find error messages. Logs tell you what happened and when.

14.2 The `/var/log` Directory

Common log files in `/var/log` : `syslog` (system events), `auth.log` (authentication), `kern.log` (kernel), `messages` (general messages, RHEL), `secure` (security), `apt/history.log` (package history), and `/var/log/nginx/` (web server logs). Application logs are often in subdirectories.

Example: Apache logs are in `/var/log/apache2/access.log` and `error.log` . MySQL logs in `/var/log/mysql/error.log` . User logs are in `/var/log/wtmp` and `lastlog` .

14.3 Journald and systemd Journal

Systemd introduced `journald`, a logging system that replaces traditional logs. The journal stores logs in a binary format. Logs are available via `journalctl`. The journal includes logs from services, kernel, and system.

Example: `journalctl` is the command to query the journal. It can show logs from a specific time, service, or with specific priorities.

14.4 Viewing Logs with journalctl

`journalctl` commands: `journalctl -f` (follow), `journalctl -u nginx` (service logs), `journalctl --since "1 hour ago"`, `journalctl -p err` (errors only), `journalctl --disk-usage` (journal size), and `journalctl -b` (boot logs).

Example:

```
# Follow new logs
journalctl -f
# Nginx service logs
journalctl -u nginx
# Logs since date
journalctl --since "2025-01-01"
# Only error and above
journalctl -p err
# Previous boot logs
journalctl -b -1
```

14.5 Log Rotation

Log rotation prevents log files from growing indefinitely. `logrotate` manages this. Configuration in `/etc/logrotate.conf` and `/etc/logrotate.d/`. It rotates logs, compresses them, and deletes old ones. Services can trigger log rotation.

Example: `/etc/logrotate.d/nginx`:

```
/var/log/nginx/*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 640 www-data adm
    postrotate
        [ -f /var/run/nginx.pid ] && kill -USR1 `cat /var/run/nginx.pid`
}
```

```
    echo "Please run as root"
    exit 1
fi

# Main script logic
log "Starting backup"
trap 'log "ERROR: Backup failed at line $LINENO"' ERR

# ... backup commands ...
log "Backup completed successfully"
```

18. Networking Services

18.1 DNS Server Basics

DNS (Domain Name System) converts domain names to IP addresses. The DNS server resolves queries. Common DNS server: BIND (Berkeley Internet Name Domain). Configuration files: `/etc/bind/named.conf` and zone files. Use `dig` and `nslookup` for testing.

Example: Install BIND: `sudo apt install bind9`. Configure a zone file for `example.com`. Test with `dig example.com`. Forwarders can be set for external resolution.

18.2 NTP Time Synchronization

NTP (Network Time Protocol) synchronizes system clocks. Time synchronization is critical for logs, authentication, and distributed systems. Common NTP client: `chrony` (newer) or `ntp` (older). Servers are listed in `/etc/chrony/chrony.conf`.

Example: Install chrony: `sudo apt install chrony`. Configure with `server 0.pool.ntp.org iburst`. Check status: `chronyc sources -v`. All servers should have synchronized time.

18.3 Web Server (Apache/Nginx)

Web servers host websites and applications. Apache (httpd) is the traditional web server. Nginx is a newer, high-performance alternative. Configuration is in `/etc/apache2/` or `/etc/nginx/`. Virtual hosts host multiple sites. SSL/TLS is essential for security.

Example: Install Nginx: `sudo apt install nginx`. Configure site in `/etc/nginx/sites-available/default`. Enable SSL: `sudo certbot --nginx -d example.com`. SSL certificates are issued by Let's Encrypt.

18.4 Database Server (MySQL/PostgreSQL)

Database servers store application data. MySQL/MariaDB and PostgreSQL are popular open-source databases. Configuration includes access control, performance tuning, and backups. Install and secure:

`mysql_secure_installation` . Create databases and users.

Example: Install MySQL: `sudo apt install mysql-server` . Secure: `sudo mysql_secure_installation` . Create database: `CREATE DATABASE mydb;` . Create user: `CREATE USER 'appuser'@'localhost' IDENTIFIED BY 'password';` .

18.5 DHCP Server

DHCP (Dynamic Host Configuration Protocol) automatically assigns IP addresses. Common DHCP server: ISC DHCP. Configuration is in `/etc/dhcp/dhcpd.conf` . Define subnet ranges, default gateway, DNS, and lease times. DHCP simplifies network management.

Example: Configure DHCP for a subnet: `subnet 192.168.1.0 netmask 255.255.255.0 { range 192.168.1.10 192.168.1.100; option routers 192.168.1.1; option domain-name-servers 8.8.8.8; }` .

18.6 File Server (NFS/Samba)

NFS (Network File System) shares files between Linux systems. Samba shares files with Windows systems. NFS configuration in `/etc/exports` . Samba configuration in `/etc/samba/smb.conf` . Both support authentication and access controls.

Example: NFS share: `/srv/nfs`
`192.168.1.0/24(rw,sync,no_subtree_check)` . Samba share: `[shared]`
`path = /srv/samba browseable = yes read only = no` .

19. Security Hardening

19.1 Security Principles

Security hardening reduces vulnerabilities. Principles: least privilege (users/services get minimum permissions), defense in depth (multiple layers of security), regular updates, and strong authentication. Hardening is an ongoing process, not a one-time task.

Example: A web server has SSH open only from internal networks, runs services as non-root users, uses TLS, has IP-based firewall rules, and keeps software updated. Each layer adds security.

19.2 System Updates

Regular system updates patch security vulnerabilities. Use automatic updates for security patches. Test updates in staging first. Monitor for new security updates. Subscribe to security mailing lists. Use tools like `unattended-upgrades`.

Example: Configure unattended-upgrades on Ubuntu: `sudo dpkg-reconfigure --priority=low unattended-upgrades`. It automatically installs security updates. On RHEL/CentOS, configure `dnf-automatic`.

19.3 Disabling Unused Services

Disable services that aren't needed. Each running service is a potential attack vector. Use `systemctl list-units --type=service --state=running` to list running services. Stop and disable unused services. Remove unnecessary packages.

Example: `sudo systemctl stop cups && sudo systemctl disable cups` disables the CUPS print service. `sudo apt remove --purge cups` removes it completely.

19.4 Security-Enhanced Linux

SELinux (Security-Enhanced Linux) is a mandatory access control system for RHEL/CentOS. It enforces security policies. Modes: enforcing (active), permissive (logs only), disabled. Use `sestatus` to check status. `audit2why` interprets denials.

Example: Set SELinux to enforcing: `sudo setenforce 1`. Manage file contexts: `sudo chcon -t httpd_sys_content_t /var/www/html`. Set boolean: `sudo setsebool -P httpd_can_network_connect on`.

19.5 AppArmor

AppArmor is a mandatory access control system for Debian/Ubuntu. It profiles applications to restrict their capabilities. Profiles are in `/etc/apparmor.d/`. Status: `sudo aa-status`. Manage profiles: `aa-enforce`, `aa-complain`, `aa-disable`.

Example: Install AppArmor utilities: `sudo apt install apparmor-utils`. Enforce a profile: `sudo aa-enforce /usr/sbin/nginx`. Place profiles in `/etc/apparmor.d/`.

19.6 User Security

User security includes: strong passwords, regular password changes, disabling inactive accounts, limiting access, and auditing user activity. Use

`passwd` for passwords, `faillog` for failed attempts, and `last` for login history. Remove unused accounts.

Example: Lock user: `sudo passwd -l john`. Force password change on next login: `sudo chage -d 0 john`. List users with expired passwords: `sudo chage -l john`.

19.7 File System Security

File system security includes: correct permissions (`chmod`), ownership (`chown`), mount options (`noexec` , `nosuid` , `nodev`), and encryption. Use `find` to find world-writable files, `sticky bit` on `/tmp`, and `umask` to set default permissions.

Example: Secure `/tmp`: `mount -o remount,noexec,nosuid,nodev /tmp`. Find world-writable files: `find / -type f -perm -o+w -ls 2>/dev/null`. Set SUID on `passwd`: `chmod u+s /usr/bin/passwd`.

20. Performance Tuning

20.1 Performance Metrics

Key performance metrics: CPU utilization, memory usage, disk I/O, network bandwidth, and load average. Establish baselines to detect anomalies. Use tools: `top` , `vmstat` , `iostat` , `sar` , and `netstat` . Monitor regularly and tune when needed.

Example: Baseline CPU at 20% idle. When idle drops to 5% consistently, investigate. Use `top` to find CPU-hungry processes. Tune by optimizing code or adding more resources.

20.2 CPU Tuning

CPU tuning includes: process scheduling, CPU affinity, and process priority. Use `nice` and `renice` to adjust process priority. CPU affinity binds processes to specific CPU cores. Use `taskset` for affinity. Kernel parameters can be tuned.

Example: Set process priority: `nice -n -20 ./high_priority`. Change priority: `renice -n 10 -p 1234`. Set CPU affinity: `taskset -c 0,1 ./process`.

20.3 Memory Tuning

Memory tuning includes: swap management, cache settings, and memory limits. Adjust `vm.swappiness` (0-100) to control swap usage. Lower values

reduce swapping. Use `ulimit` to set memory limits. Tune file system cache with `vm.vfs_cache_pressure`.

Example: Set swappiness to 10: `sysctl vm.swappiness=10`. Make permanent in `/etc/sysctl.conf`. Set memory limit for user: `ulimit -m 1024000`.

20.4 Disk I/O Tuning

Disk I/O tuning includes: filesystem selection (ext4, xfs), mount options (`noatime`, `nodiratime`), scheduler (CFQ, noop, deadline), and RAID. Use `iostat` to monitor I/O. Tune filesystem parameters.

Example: Mount with `noatime` for performance: `mount -o noatime /dev/sda1 /`. Change I/O scheduler: `echo deadline > /sys/block/sda/queue/scheduler`.

20.5 Network Tuning

Network tuning includes: TCP parameters (window size, buffer sizes), interface settings (MTU, offloading), and congestion control. Use `ifconfig` or `ip` to check stats. Tune `net.core` and `net.ipv4` kernel parameters.

Example: Increase TCP buffer size: `sysctl -w net.core.rmem_max=16777216`. Set in `/etc/sysctl.conf`. Change congestion control: `sysctl -w net.ipv4.tcp_congestion_control=bbr`.

20.6 Kernel Tuning

Kernel tuning involves modifying `/etc/sysctl.conf`. Parameters control memory, network, filesystem, and scheduling behavior. Apply with `sysctl -p`. Test changes in a non-production environment. Tune based on workload.

Example: Parameters for high-performance web server:

```
net.core.somaxconn=65535
net.ipv4.tcp_tw_reuse=1
vm.vfs_cache_pressure=50
kernel.sem=250 32000 100 128
```

21. Troubleshooting

21.1 Troubleshooting Methodology

A systematic approach: identify the problem, gather information, reproduce the issue, isolate the cause, implement a fix, and verify the resolution.

Document findings. Use logs, metrics, and tests. Start with the obvious (is the service running? is the network up?).

Example: Users report a website is slow. Check: is the server responding? CPU/memory? Is the database slow? Network bandwidth? Check logs. Test with `curl`. Find the bottleneck. Resolve and test.

21.2 Boot Issues

Boot issues: system won't boot, kernel panic, filesystem errors. Use rescue mode (boot from install media). Check `/var/log/boot.log`. Test filesystem with `fsck`. Reinstall boot loader (GRUB). Recover from backups.

Example: System fails to boot with "kernel panic." Boot from rescue CD. Mount root partition. Check `/var/log/boot.log`. Repair with `fsck /dev/sda1`. Reinstall GRUB: `grub-install /dev/sda`.

21.3 Application Issues

Application issues: service won't start, crashes, errors in logs. Check service status: `systemctl status app`. Check logs in `/var/log/app/`. Verify configuration syntax. Test dependencies (database, network). Check disk space and permissions.

Example: Nginx fails to start. Check `systemctl status nginx`. Check logs: `tail /var/log/nginx/error.log`. Verify config: `nginx -t`. Fix syntax error, reload config, restart service.

21.4 Network Issues

Network issues: connectivity problems, DNS failures, slow performance. Start with `ping`, `traceroute`, and `curl`. Check DNS with `dig`. Verify firewall rules (`iptables -L`). Check routing (`ip route`). Use `tcpdump` for packet analysis.

Example: Cannot ping server. Check `ifconfig` (is interface up?). Check `route` (is default gateway set?). Check `iptables -L` (is ICMP blocked?). Use `traceroute` to find where traffic stops.

21.5 Permission Issues

Permission issues: "Permission denied" errors, cannot write files, cannot access directories. Check ownership (`ls -l`). Check permissions (`chmod`, `chown`). Verify user belongs to required groups (`groups`). Check SELinux/AppArmor.

Example: User cannot write to `/var/log/app.log`. Check `ls -l /var/log/app.log` (ownership/permissions). Check `id john` (user groups).

Check `selinuxenabled` (SELinux status). Fix with `chown` or `chmod`.

21.6 Resource Issues

Resource issues: out of memory, high CPU, disk full. Check `free -m` (memory), `df -h` (disk), `top` (CPU). Kill idle processes or add more resources. Clean up old logs. Resize partitions or add more storage.

Example: Disk full. `df -h` shows 100% used. `du -sh /var` finds large directories. Clean logs, remove temp files, or expand filesystem. Add new disk if needed.

22. Virtualization

22.1 Virtualization Overview

Virtualization runs multiple virtual machines (VMs) on a single physical host. It improves resource utilization. Popular hypervisors: KVM (Linux), VMware, Xen. Containers (Docker) are lighter than VMs. Virtualization enables testing, isolation, and legacy support.

Example: A server with 64GB RAM runs 10 VMs: 4 web servers, 3 database servers, 2 application servers, and 1 monitoring VM. Resources are allocated dynamically.

22.2 KVM Virtualization

KVM (Kernel-based Virtual Machine) is the Linux kernel virtualization module. Management tools: `virsh`, `virt-manager`. VMs are stored as disk images. KVM supports live migration, snapshots, and resource management.

Example: Install KVM: `sudo apt install qemu-kvm libvirt-daemon-system virtinst virt-manager`. Create VM: `virt-install --name web-server --ram 2048 --disk path=/var/lib/libvirt/images/web.img,size=20 --vcpus 2 --os-type linux`.

22.3 Managing Virtual Machines

`virsh` manages VMs. Commands: `virsh list` (running VMs), `virsh start` (start VM), `virsh shutdown` (shutdown VM), `virsh destroy` (force stop), `virsh console` (connect to console), `virsh snapshot-create` (snapshot), and `virsh dumpxml` (export XML configuration).

Example:

```
# List all VMs
sudo virsh list --all
# Start VM
sudo virsh start web-server
# Shutdown VM
sudo virsh shutdown web-server
# Connect to console
sudo virsh console web-server
# Create snapshot
sudo virsh snapshot-create web-server
```

22.4 Docker Basics

Docker is a containerization platform. It runs applications in isolated containers. Containers share the host kernel but have their own filesystem and network. Docker images are built from Dockerfiles. Containers are lightweight and fast to start.

Example: Install Docker: `sudo apt install docker.io`. Run a container: `docker run -d --name web -p 80:80 nginx`. Build a custom image: `docker build -t myapp .` Manage containers: `docker ps`, `docker stop`, `docker rm`.

22.5 Container Management

Docker commands: `docker run` (create and start container), `docker ps` (list containers), `docker images` (list images), `docker stop` (stop container), `docker rm` (remove container), `docker rmi` (remove image), `docker exec` (run command in container), `docker logs` (view logs).

Example:

```
# List all containers
docker ps -a
# List images
docker images
# Stop container
docker stop web
# Remove container
docker rm web
# Bash shell in container
docker exec -it web bash
# View logs
docker logs web
```

23. System Updates and Upgrades

23.1 Update Strategy

An update strategy defines when and how to apply updates. Include: security updates (immediate), minor updates (weekly), major upgrades (planned). Test in staging first. Maintain rollback plan. Schedule maintenance windows. Monitor after updates.

Example: Security patches are applied automatically daily. Minor package updates are tested on staging Monday, applied to production Wednesday. Major version upgrades are planned quarterly with thorough testing.

23.2 Security Updates

Security updates fix vulnerabilities. They're critical and should be applied as soon as possible. Use `unattended-upgrades` on Debian/Ubuntu, `dnf-automatic` on RHEL. Monitor security mailing lists. Apply updates in staging first to test compatibility.

Example: Debian/Ubuntu `unattended-upgrades` configuration. RHEL `dnf-automatic` configuration. Check for security updates: `apt list --upgradable | grep -i security`. RHEL: `dnf update --security`.

23.3 Upgrading Packages

Package upgrades install new versions of software. Use `apt upgrade` or `dnf update`. `dist-upgrade` handles major version changes. Review upgrade notes. Check for deprecated features. Test after upgrade.

Example: Ubuntu: `sudo apt update && sudo apt upgrade -y`. RHEL: `sudo dnf update -y`. After upgrade, verify services are running correctly. Check logs for errors.

23.4 Upgrading Distributions

Distribution upgrades move to a new major version (e.g., Ubuntu 22.04 to 24.04). They require careful planning. Backup data. Read release notes. Test the upgrade on a staging system. Handle with `do-release-upgrade` on Ubuntu, `dnf system-upgrade` on RHEL.

Example: Ubuntu: `sudo do-release-upgrade`. RHEL: `sudo dnf install dnf-plugin-system-upgrade && sudo dnf system-upgrade download --releasever=9 && sudo dnf system-upgrade reboot`.

23.5 Managing Updates with Unattended-Upgrades

`unattended-upgrades` automates security updates on Debian/Ubuntu. Configure it to automatically install security updates. It can email reports. It reduces administrative overhead. Review logs weekly.

Example: Install: `sudo apt install unattended-upgrades` . Configure: `sudo dpkg-reconfigure --priority=low unattended-upgrades` . Check status: `sudo systemctl status unattended-upgrades` .

24. Conclusion

Congratulations! You've completed this comprehensive guide to Linux Administration. Here's a summary of what you've learned:

- Linux administration involves managing, maintaining, and configuring Linux systems for performance, security, and reliability.
- The Linux filesystem follows the Filesystem Hierarchy Standard with key directories like `/etc` , `/var` , `/home` , and `/tmp` .
- User and group management controls access through `/etc/passwd` and `/etc/shadow` .
- File permissions and ownership use read/write/execute with numeric (755) and symbolic representations.
- Package management with APT (Debian) and YUM/DNF (RHEL) installs and updates software.
- System services and processes are managed with `systemctl` and process management commands.
- System monitoring tracks CPU, memory, disk, and network performance.
- Disk management covers partitioning, formatting, mounting, and LVM.
- Firewall management with iptables, UFW, and firewalld controls network access.
- SSH provides secure remote access with key-based authentication.
- System logs in `/var/log` and journalctl are essential for troubleshooting.
- Backup strategies using tar, rsync, and cron ensure data protection.
- Cron schedules automated tasks with precise timing.
- Shell scripting automates repetitive tasks with variables, conditionals, and loops.
- Security hardening involves updates, disabling services, SELinux/AppArmor, and user security.
- Performance tuning optimizes CPU, memory, disk, and network resources.
- Troubleshooting uses a systematic approach to identify and resolve issues.
- Virtualization with KVM and Docker enhances resource utilization and isolation.

- System updates require a strategic approach for security and stability.